PropManage

Beginner-Friendly Viva Master Guide

Property Management Application using Power BI

Duppati Ashritha | 24WH1DB008

Guide: Mr. Ch. Anil Kumar, Assistant Professor, Dept. of Information Technology

Everything explained in plain, simple English — no assumed prior knowledge required.

Welcome — How to Use This Guide

This guide explains your entire project, PropManage, in very simple language, as if you are learning about it for the very first time. You do not need to memorise this guide word for word. Instead, read it two or three times so that the ideas feel natural to you, and then practise saying them in your own words. The examiner is not trying to trap you; they simply want to see that you understand what you built and why you built it the way you did. This guide is organised into four parts: a simple explanation of every part of your project, ready-made speaking scripts of different lengths, one hundred practice questions with simple answers, and a final one-page revision sheet you can glance at right before you walk in.

Part A — Your Project Explained Simply

What is PropManage, in one simple sentence?

PropManage is a website-based system that helps property owners and renters do everything in one place — search for a property, book a visit, pay money safely, sign a contract, ask for repairs, and even talk to an AI assistant using their voice — instead of doing all of this manually through phone calls and paper.

1. Abstract — The Big Picture

Think of the Abstract as the trailer of a movie — it tells you the whole story in a few lines before the details begin. In simple words, small property businesses today still use spreadsheets, WhatsApp messages, and paper files to manage their properties, and this creates mistakes and delays. We built PropManage using four technologies that work together — MongoDB, Express, React, and Node, together called the MERN stack — to put everything into one clean system. This system also checks payments carefully so nobody can cheat, sends instant alerts to staff, and lets users talk to an AI assistant in English, Hindi, or Telugu. We proved it works by running thirty-eight functional tests and fourteen security tests, and all of them passed.

Keywords to remember: *MERN stack, fragmented workflow, verified payments, multilingual voice assistant, Power BI*

2. Introduction — Why This Problem Matters

In simple terms, the Introduction explains why we picked this problem in the first place. Imagine banks and shopping websites today are all digital, but many small property businesses still write down bookings on paper. This causes real problems, like two families being told they can visit the same house at the same time, which we call double-booking. We wanted to build something that fixes this using modern web technology, while also making sure people who are not comfortable speaking English can still use the system easily, using their own voice in Hindi or Telugu.

Keywords to remember: *digital gap, double-booking, accessibility, non-English speakers*

3. Problem Statement — The Exact Problem We Are Solving

The Problem Statement is simply a clearer, more specific version of the Introduction. In plain words, small property operators do not have one single system that handles everything from listing a property to collecting the final maintenance request. Because everything is manual, five bad things keep happening: visits get double-booked, nobody can prove a payment actually happened, staff find out about new bookings late because they have to keep checking their inbox, users who only speak a regional language get no help outside office hours, and owners have no way to see live business reports on their own.

Keywords to remember: *lifecycle gap, unverifiable payments, inbox polling, no BI view*

4. Objectives — What We Promised to Build

Objectives are simply the list of promises we made to ourselves before starting to code, and later we check if we kept every promise. We promised to put the whole property process in one system, to alert staff instantly instead of making them wait, and to check every big payment using a special mathematical proof called a signature, so nobody can fake a payment. We also promised to build a voice assistant that understands three languages, to

make sure a blocked user is locked out immediately and not just eventually, to give owners a live business dashboard through Power BI, and finally to test everything properly instead of just assuming it works.

Keywords to remember: *sub-second alerts, HMAC signature, RBAC, Power BI dashboard, empirical testing*

5. Scope of Work — Where We Drew the Line

Scope of Work simply means deciding what we will build now and what we will build later, so the project stays realistic. Inside our scope, we built property search, visit and purchase booking, UPI and Razorpay payments, rental and purchase contracts, maintenance requests, instant admin alerts, the voice assistant, and Power BI reports. We deliberately left out a mobile phone app, full Arabic right-to-left screen support, smart lock devices, and price-prediction features, because trying to do everything at once usually means doing nothing well. These leftover ideas are not forgotten — they appear again later under Future Scope.

Keywords to remember: *in scope vs out of scope, deliberate boundary, future scope link*

6. Literature Review — What Other Researchers Found

A Literature Review simply means reading what other smart people have already studied about our topic, so we do not repeat mistakes or ignore useful ideas. We studied six research papers. Some told us that small property businesses are still stuck using spreadsheets, some told us that the MERN stack we chose actually saves developers a lot of extra work, and some told us that AI chatbots are genuinely helpful but usually only work in English, leaving out Hindi and Telugu speakers. Other papers showed that login-token security, called JWT, often has hidden bugs if not implemented carefully. Reading all of this helped us understand exactly what problems were still unsolved, which we explain next.

Keywords to remember: *research papers, MERN benefit, English-only chatbot gap, JWT security bugs*

7. Research Gap — What Nobody Had Solved Yet

A Research Gap simply means finding the missing piece that nobody else has built yet. After reading all those papers, we noticed three missing pieces. First, nobody tested a full real-world system combining everything together, they only tested small pieces separately. Second, security studies checked login tokens like JWT only as a lab experiment, not inside a real, working, attacked system. Third — and this is our biggest opportunity — nobody had built one single assistant that understands your own data, speaks multiple languages, AND listens and talks back using voice, all without spending extra money on expensive speech services. PropManage was built specifically to fill all three missing pieces at once.

Keywords to remember: *missing piece, isolated testing, our unique combination, zero extra cost*

8. Existing System — What People Use Today

The Existing System section simply describes what is already available in the market before our project. Today, people either coordinate purely by phone and paper, or they use listing websites like Bayut and Dubizzle, which only let you browse pictures but do not let you book a visit or pay money through the website. There are also big paid software packages like Buildium and AppFolio which do more, but they are expensive and still do not offer voice assistants or instantly-verified payments. In short, every existing option is either too basic, too manual, or too expensive for a small property business.

Keywords to remember: *browsing-only portals, expensive enterprise suites, no voice/no verified payments*

9. Proposed System — Our Solution

The Proposed System is simply our answer to everything wrong with the existing systems. We built one single website system with two separate sections, one for normal users and one for administrators, so each side sees only what it needs to see. Inside this system, staff get instant alerts the moment something happens, payments are checked using a cryptographic proof instead of just trusting what the user says, and users can speak to an AI assistant in English, Hindi, or Telugu using just their web browser's built-in microphone, at no extra cost. The most special part of our project, the one thing no other system combines, is this voice assistant that actually knows live information about real properties and bookings, not just generic answers.

Keywords to remember: *dual-portal, instant alerts, cryptographic proof, zero-cost voice, live-data-aware AI*

10. System Architecture — How the Pieces Fit Together

Think of System Architecture as a map showing how information travels through our system, like a delivery route. First, the user opens the website, called the Frontend, built using a technology called React. Before doing anything important, the system checks who the user is, which we call Authentication, using something called a JWT token, similar to a stamped entry pass. If the user wants to talk to the AI assistant, that request also passes through this same security gate before reaching Google's Gemini AI service. Every request then reaches the Backend, built using Node.js and Express, which is like the kitchen of a restaurant that actually processes the order. The backend then talks to the Database, called MongoDB, where all information is safely stored, and finally, the same backend also feeds information to the Admin Panel and to Power BI, so administrators and business reports always see the exact same live data, not an old or separate copy.

Keywords to remember: *Frontend=React, Authentication=JWT, Backend=Node/Express, Database=MongoDB, single source of truth*

11. Algorithms / Methodology — The Five Rules Our System Follows

An algorithm is simply a fixed set of steps a computer follows every single time, like a recipe. Our first rule is login security: every time a user does something, the system re-checks their identity and their status, so if an account is blocked, it loses access immediately, not after some delay. Our second rule is payment checking: instead of trusting whatever the payment app tells us, our own server independently recalculates a secret mathematical code, called HMAC-SHA256, and only accepts the payment if this code matches exactly, which stops anyone from faking a successful payment. Our third rule is instant alerts: the moment a booking or payment is saved, the server immediately pushes a notification to the admin, with no waiting or refreshing needed. Our fourth rule is the voice assistant pipeline: your voice is converted to text by the browser itself, sent to Google's Gemini AI along with real information about our properties, and the AI's answer is read back aloud to you in your chosen language. Our fifth rule is the Power BI pipeline, where the database itself does all the heavy counting and grouping work, so Power BI stays fast even as data grows.

Keywords to remember: *per-request re-check, HMAC-SHA256, instant push, speech-to-text-to-speech, database-side aggregation*

12. Working Modules — The Seven Teams Inside Our System

A module is simply one clearly separated part of the system responsible for one job, like different departments inside a company. The User Module lets people search properties and make bookings. The Authentication Module checks who everyone is and what they are allowed to do. The Voice Assistant Module handles all the speaking and listening using Gemini AI. The Booking and Payment Module manages reservations and money through Razorpay.

The Admin Module lets staff approve requests and manage records. The Database Module, built on MongoDB, is where everything is permanently stored. And the Reporting Module connects our data to Power BI so owners can see visual reports. Splitting the project this way means each part can be fixed or improved without breaking the others.

Keywords to remember: *separation of responsibility, seven modules, independent maintenance*

13. Database Explained Simply — What Is Actually Stored

Our project does not use old-style spreadsheet-like tables the way traditional databases such as MySQL do. Instead, we use MongoDB, which stores information as flexible documents, similar to individual index cards, grouped into folders called collections. In simple terms, we have a Users collection holding login details for people using the site, a Properties collection holding every listed house or apartment with its price, location, and photos, and a Bookings collection recording every visit or purchase request along with its date and status. We also have a Payments collection storing every transaction along with its verified signature, a Contracts collection tracking rental and purchase agreements through their entire lifecycle, and separate collections for Owners, Tenants, financial transactions, service or maintenance requests, and notifications. Choosing MongoDB over a traditional table-based database made sense because our data, like a property with many different features, does not always fit neatly into fixed rows and columns.

Keywords to remember: *MongoDB = documents not tables, collections, Users/Properties/Bookings/Payments/Contracts*

14. APIs Explained Simply — The Messengers of Our System

An API is simply a fixed way for one part of a program to ask another part for something, like ordering food using a menu instead of walking into the kitchen yourself. Our Authentication API handles login, registration, and password reset. Our Properties API lets users search and filter listings. Our Bookings API lets users request a visit or purchase and lets staff confirm or reject it. Our Payments API handles both manual UPI payments and automatic Razorpay payments, always checking the cryptographic signature before accepting anything. Our Purchase Contracts API manages the advance payment and digital signing process for buying a property. Our AI Chat API is the doorway to our voice assistant, and finally, our Analytics API, protected by its own special access key, feeds live numbers into Power BI without ever exposing our main user login system.

Keywords to remember: *API = request doorway, Auth/Properties/Bookings/Payments/Contracts/AI/Analytics APIs*

15. Technologies Used — And Why We Chose Each One

For the Frontend, the part users actually see and click on, we chose React.js because it lets us build fast, reusable pieces of the screen instead of rewriting the same code again and again. For the Backend, the part that does the actual thinking and processing, we chose Node.js with Express.js because both the frontend and backend can then be written in the same programming language, JavaScript, which saves time and reduces mistakes. For the Database, we chose MongoDB Atlas, a cloud-hosted version of MongoDB, because our data has many flexible, connected pieces that suit a document-style database better than a rigid table-style one. For real-time alerts, we used a tool called Socket.IO, which keeps a live connection open so messages arrive instantly instead of the browser having to keep asking, “any updates yet?” For payments, we used Razorpay because it is a trusted, widely used gateway with strong security documentation. For AI and voice, we used Google's Gemini API alongside the browser's own built-in Web Speech API, which means we get speech recognition and speech output completely free, without paying for a separate voice service. Finally, for business reporting, we connected everything to Power BI, a well-known Microsoft tool that turns raw numbers into easy-to-read charts and dashboards.

Keywords to remember: *React=frontend, Node+Express=backend, MongoDB Atlas=database, Socket.IO=real-time, Razorpay=payments, Gemini+Web Speech API=free voice AI, Power BI=reporting*

16. Complete Workflow — A Real-Life Example

Let me explain the entire project using one simple, real-life story. Imagine a woman named Priya wants to rent a two-bedroom apartment. She opens the PropManage website, searches by city and budget, and finds a flat she likes. Because Priya is more comfortable speaking Telugu, she taps the microphone button and asks the voice assistant, in Telugu, whether this flat allows pets, and the assistant checks the real property details and answers her back, out loud, in Telugu. Satisfied, Priya books a visit for Saturday. The moment she submits this booking, our system instantly pushes a notification to the property admin's screen, without the admin having to refresh the page even once. The admin confirms the visit, Priya likes the flat in person, and later pays her advance amount through Razorpay. Behind the scenes, our server does not just trust that the payment succeeded — it independently recalculates a secret security code and only marks the payment as verified once that code matches exactly. Finally, the property owner logs into Power BI and instantly sees this new booking and payment reflected in their live business dashboard, without asking anyone or waiting for a report to be emailed to them.

Keywords to remember: *real user journey, voice search, instant admin alert, verified payment, live owner dashboard*

17. Diagrams Explained Simply

Our System Architecture diagram is simply a left-to-right flow showing how a request travels: from the User, through the Frontend screen, through the Authentication security check, optionally through the Voice Assistant, into the Backend, then into the Database, and finally out to the Admin Panel and Power BI Reports. Our Sequence diagram, shown for the booking process, is simply a step-by-step timeline read from top to bottom: the user sends a booking request, the backend saves it in the database, the database confirms it was saved, the backend creates a notification, and that notification is instantly pushed to the admin's screen, all within about two seconds, while the user separately receives a confirmation message. Reading any such diagram simply means following the arrows in order and reading each label as one small step in the story.

Keywords to remember: *architecture = left-to-right map, sequence diagram = top-to-bottom timeline*

18. Testing and Results — Proving It Actually Works

Testing simply means deliberately trying to break your own system before someone else does. We ran thirty-eight functional tests, which check normal, expected use, like registering an account, searching properties, or signing a contract, and every single one of these thirty-eight tests passed. We also ran fourteen adversarial security tests, which means we deliberately tried bad or dishonest actions, such as using an expired login token, uploading a fake image file, or trying to fake a payment signature, and all fourteen of these attacks were correctly blocked. Separately, we tested performance by simulating one hundred people using the website at exactly the same time, and the system still responded in under half a second on average, with less than 0.2 percent of requests failing, which shows the system stays reliable even under real pressure, not just when nobody is using it.

Keywords to remember: *38/38 functional pass, 14/14 security pass, 100 concurrent users, sub-500ms latency*

19. Advantages, Limitations, and Future Enhancements

In simple terms, the biggest advantage of PropManage is that it brings together features that normally only exist separately, at a very low cost, since almost every tool we used has a free tier. However, we are also honest about its limitations: right now, if the admin dashboard tries to load eight different reports at once and even one of them

fails, the whole dashboard can fail along with it, which is something we plan to fix. Our voice assistant also uses a simpler method of feeding it live data rather than a more advanced technique called retrieval-augmented generation, or RAG, which would help it scale better as our property list grows much larger. Looking forward, our future enhancements include building a proper mobile app, adding full right-to-left Arabic language support, predicting rental prices using past data, and eventually connecting smart locks and IoT devices for keyless property access.

Keywords to remember: *low-cost advantage, dashboard single-point failure, future RAG upgrade, mobile app & IoT future work*

Part B — Ready-Made Speaking Scripts

The 1-Minute Explanation

Good morning. My project is called PropManage, a Property Management Application using Power BI. Small property businesses today still manage everything manually, using spreadsheets and phone calls, which causes double-bookings and payment disputes. I built a MERN stack website that brings property listing, booking, payments, contracts, and maintenance into one system. It sends instant alerts to staff, verifies every payment using a cryptographic signature, and includes a multilingual voice assistant that works in English, Hindi, and Telugu using the browser's own free speech tools. I also connected the system to Power BI so property owners get live business dashboards. I tested everything thoroughly, and all thirty-eight functional tests and all fourteen security tests passed successfully.

The 3-Minute Explanation

Good morning. My project is called PropManage, a Property Management Application using Power BI, and it solves a real problem: small and mid-size property businesses still coordinate everything manually, through phone calls, spreadsheets, and paper records, which leads to double-booked visits and payments that nobody can properly verify. After studying existing research and existing tools like Bayut, Dubizzle, Buildium, and AppFolio, I found that none of them combine verified payments, real-time alerts, and voice-based AI assistance in one affordable system, and that became the specific gap my project fills.

I built PropManage using the MERN stack, meaning MongoDB for the database, Express and Node.js for the backend, and React for two separate frontend portals, one for regular users and one for administrators. Whenever a booking or payment happens, the system instantly notifies admin staff using a technology called Socket.IO, removing the delay of manually checking for updates. For payments, instead of blindly trusting the payment gateway's response, my server independently recalculates a security signature using HMAC-SHA256 and only accepts the payment if it matches exactly, which protects against fraud.

The most unique part of my project is the multilingual voice assistant. Using the browser's own built-in speech recognition and speech synthesis, combined with Google's Gemini AI, users can literally speak to the system in English, Hindi, or Telugu, and get spoken answers grounded in real, live property data, all without needing any separate paid speech service. I also built a dedicated, secured analytics API so property owners can view live Power BI dashboards. Finally, I tested the whole system thoroughly: all thirty-eight functional test cases passed, all fourteen security attack tests were correctly blocked, and the system handled one hundred simultaneous users with an average response time under half a second.

The 5-Minute Explanation

Good morning, respected panel. My project, PropManage, subtitled a Property Management Application using Power BI, addresses a very practical and widespread problem. Despite banking, hospitality, and e-commerce having become fully digital over the last decade, small and mid-size property management businesses still rely heavily on manual coordination, spreadsheets, phone calls, and paper documentation. This leads to real, damaging consequences: double-booked property visits, payment records nobody can independently verify, staff who only discover new bookings after manually checking their inbox, and no

assistance at all for customers outside office hours or those who are not comfortable communicating in English.

Before designing my solution, I reviewed six research papers spanning MERN architecture, AI chatbot performance, multilingual voice assistant challenges, and JWT authentication security. This review revealed three specific gaps: architecture and security techniques are usually studied in isolation rather than inside one real, working system; and most importantly, no existing system combines domain-aware AI, multiple languages, and voice interaction together at zero additional infrastructure cost. I compared PropManage directly against existing options, including listing portals like Bayut and Dubizzle and paid enterprise suites like Buildium and AppFolio, and none of them offer verified real-time payments or AI voice assistance at all.

My proposed system is a role-segmented, dual-portal MERN stack application, meaning one portal for regular users and a separate, secured portal for administrators, both talking to one shared backend and database so nobody sees inconsistent data. The system architecture flows from the user, through the React frontend, through JWT-based authentication and role checking, optionally through the Gemini-powered voice assistant, into the Node.js and Express backend, into the MongoDB Atlas database, and finally out to both the admin panel and a dedicated, independently secured Power BI analytics API.

Technically, five mechanisms do the real work. Authentication re-checks a user's role and active status on every single request, so a blocked account is locked out immediately. Payments are verified using an HMAC-SHA256 cryptographic signature recomputed independently on the server, making client-side tampering impossible. Real-time notifications use Socket.IO to push alerts to administrators the instant an event happens, with zero polling delay. The voice assistant pipeline captures speech through the browser's free, built-in Web Speech API, sends a language-conditioned, data-grounded prompt to Gemini, and reads the reply back aloud. And the Power BI pipeline performs all aggregation directly inside MongoDB, so performance scales with reporting complexity rather than raw data volume.

To validate all of this, I tested the system against a realistic dataset of forty-five properties and one hundred and twenty users. All thirty-eight functional test scenarios passed, all fourteen adversarial security test cases behaved exactly as expected, and under simulated load of one hundred concurrent users, the system maintained an average response time under five hundred milliseconds with less than 0.2 percent errors. In conclusion, PropManage proves that a small property business can access enterprise-grade features, verified payments, real-time alerts, multilingual voice AI, and live business intelligence, entirely using free-tier technology, and I have outlined clear future enhancements including retrieval-augmented AI grounding, a native mobile app, and predictive analytics. Thank you.

If the Examiner Says: “Explain Your Complete Project”

If you are asked to explain everything in full detail with no time limit, simply follow the order of Part A of this guide from beginning to end: start with the Abstract and Introduction to set context, move through the Problem Statement, Objectives, and Scope, then explain the Literature Review and Research Gap to show you understand prior work, walk through the Existing System versus your Proposed System, explain the System Architecture and the five Algorithms in your own words using the real-life Priya example, describe the seven Working Modules and how the database and APIs support them, and finish with your Testing Results,

Advantages, Limitations, and Future Scope. Speaking in this exact order automatically makes you sound organised and confident, because it mirrors the natural flow of a real research project.

Part C — 100 Viva Questions with Simple Answers

These one hundred questions are grouped by category. Read each answer once, then try answering out loud in your own words without looking. The keywords listed are the words you must not forget, even if you forget the full sentence.

General / Conceptual Questions

Q1. What is your project about?

My project, PropManage, is a website system that helps property owners and renters manage listings, bookings, payments, contracts, and maintenance all in one place, instead of doing everything manually.

Keywords: unified platform, property lifecycle

Q2. What problem does your project solve?

It solves the problem of small property businesses relying on manual, paper-based coordination, which causes double-bookings, unverifiable payments, and delayed communication.

Keywords: manual coordination, double-booking

Q3. Who are the users of your system?

There are three types of users: property seekers or tenants who search and book, property owners who list properties, and administrators who manage everything from the admin portal.

Keywords: three user roles

Q4. What makes your project different from a normal property listing website?

Unlike simple listing sites, my project verifies payments cryptographically, sends real-time alerts, and includes a multilingual voice assistant, none of which typical listing sites offer.

Keywords: verified payments, real-time, voice AI

Q5. What is the full form of MERN?

MERN stands for MongoDB, Express.js, React.js, and Node.js, the four technologies used together to build the entire application.

Keywords: MongoDB, Express, React, Node

Q6. Why did you choose the property management domain?

I chose this domain because it is a real, practical problem where small businesses are still stuck with manual processes, unlike banking or e-commerce which have already digitised.

Keywords: real-world relevance, digital gap

Q7. Is this a group project or an individual project?

Answer this honestly based on your actual submission records.

Keywords: your actual answer

Q8. What was your specific role in building this project?

State your actual contribution honestly, for example handling the frontend, backend, database design, security, and testing.

Keywords: your actual contribution

Q9. What is the title of your project and why?

The title is Property Management Application using Power BI, chosen because the project combines full property lifecycle management with a dedicated business intelligence layer built on Power BI.

Keywords: title reflects two pillars

Q10. Can you summarise your project in one sentence?

PropManage unifies property management into one secure, real-time, AI-voice-assisted platform accessible without an enterprise budget.

Keywords: one-line summary

Technical Questions

Q11. What is a three-tier architecture?

A three-tier architecture simply divides a system into three layers: the presentation tier that users see, the application tier that processes logic, and the data tier that stores information, keeping each layer separate and manageable.

Keywords: presentation, application, data tier

Q12. What is JWT and how does it work in your project?

JWT, or JSON Web Token, is a digitally signed pass given to a user after login; my system checks this token on every request and also re-checks the user's live status, so access can be revoked instantly.

Keywords: JWT, signed token, per-request check

Q13. What is HMAC-SHA256 and why did you use it?

HMAC-SHA256 is a cryptographic formula that creates a unique code from secret data; I use it to independently verify that a payment truly matches what Razorpay processed, preventing fake payment claims.

Keywords: cryptographic signature, payment proof

Q14. What is WebSocket and why is it used here?

WebSocket is a technology that keeps a live, constantly-open connection between the browser and server, allowing instant two-way communication, which I use through Socket.IO for real-time admin notifications.

Keywords: live connection, Socket.IO

Q15. What is an API?

An API, or Application Programming Interface, is simply a defined way for one part of software to request information or actions from another part, like ordering from a menu instead of entering the kitchen.

Keywords: defined request interface

Q16. What is the difference between SQL and NoSQL databases?

SQL databases store data in fixed rows and columns like a spreadsheet, while NoSQL databases like MongoDB store flexible documents grouped into collections, which suits data that doesn't always have the same structure.

Keywords: fixed tables vs flexible documents

Q17. Why did you choose MongoDB over MySQL?

I chose MongoDB because property-related data, like features and amenities, varies a lot between records, and MongoDB's flexible document model handles this better than rigid SQL tables.

Keywords: flexible schema, document model

Q18. What is Mongoose?

Mongoose is a library that sits between our Node.js code and MongoDB, making it easier to define data structures and interact with the database in a structured way.

Keywords: ODM, structure layer over MongoDB

Q19. What is REST API?

A REST API is a common style of building APIs using standard web requests like GET, POST, PUT, and DELETE, which is exactly the style my backend uses to expose all its features.

Keywords: GET/POST/PUT/DELETE, REST style

Q20. What is role-based access control, or RBAC?

RBAC means giving different permissions to different types of users based on their role, so a regular user, an admin, and a super admin can each do different things inside the system.

Keywords: roles, permissions, hierarchy

Q21. What is the Web Speech API?

The Web Speech API is a free, built-in browser feature that can convert speech to text and text back to speech, which I used to build my voice assistant without paying for a separate speech service.

Keywords: browser-native, free speech tool

Q22. What is Gemini API?

Gemini is Google's large language model API, which I use on the server side to generate intelligent, context-aware replies for my chatbot and voice assistant.

Keywords: Google's AI model, server-side call

Q23. What does 'context-grounded' mean for your AI assistant?

It means the AI's answers are based on real, live data from our own database, such as actual property details, rather than the AI just guessing or making things up.

Keywords: grounded in live data, not generic

Q24. What is Power BI and why did you integrate it?

Power BI is a Microsoft tool that turns raw data into visual charts and dashboards; I integrated it so property owners can see live business insights without manually creating reports.

Keywords: visual dashboards, business intelligence

Q25. How does Power BI get data from your system?

I built six dedicated, read-only API endpoints that Power BI can call on a schedule, protected by their own special access key rather than the normal user login system.

Keywords: dedicated API endpoints, own access key

Scenario-Based Questions

Q26. What happens if a user tries to log in after their account has been blocked by the admin?

Because my system re-checks the user's active status on every single request rather than just trusting the login token, the blocked user is denied access on their very next request, not just at their next login.

Keywords: per-request re-check, immediate block

Q27. What happens if someone tries to fake a successful payment?

The server does not trust the client's claim; it recomputes the HMAC-SHA256 signature independently, and if it does not match exactly, the payment is rejected and no financial record is changed.

Keywords: recomputed signature, reject mismatch

Q28. What happens if two users try to book the same property visit at the same time?

My system checks for duplicate bookings before confirming a new one, and administrators are instantly notified so they can manage overlapping requests quickly rather than discovering them late.

Keywords: duplicate check, instant notification

Q29. What if the voice assistant doesn't understand the user's browser?

If a browser like Firefox does not support the Web Speech API, the microphone button is disabled with a clear message instead of failing silently or confusing the user.

Keywords: graceful fallback, browser support check

Q30. What happens if the AI provider (Gemini) is temporarily unavailable?

This is a genuine limitation I would explain honestly — currently the system depends on Gemini being available; adding a fallback provider is a reasonable future improvement, not something already built.

Keywords: honest limitation, future improvement

Q31. What if 500 users try to use the system at the same time?

Based on my load testing trend, error rates rise noticeably beyond 150 users, so at 500 users the system would need horizontal scaling of the server and MongoDB Atlas auto-scaling to maintain performance.

Keywords: scaling needed beyond 150 users

Q32. What if the admin dashboard's database query fails partway through?

This is a known limitation: the dashboard uses one combined batch of eight queries, so if even one fails, the entire dashboard request currently fails, which is why fault-tolerant aggregation is listed as future work.

Keywords: single batch limitation, admitted honestly

Q33. A user speaks in Hindi but wants property names left untranslated — how is this handled?

The system is specifically instructed to translate only the conversational parts of the reply into the chosen language, while leaving property names, contact numbers, and addresses exactly as they are.

Keywords: selective translation, proper nouns preserved

Q34. What if a tampered file is uploaded as a property image?

The system validates uploaded files, and a test specifically confirmed that an executable file disguised as an image is correctly rejected rather than accepted.

Keywords: upload validation, security tested

Q35. How would you handle a sudden spike of fake bookings, or a bot attack?

Rate limiting is already in place to block excessive repeated requests within a short time window, which is one of the fourteen security tests my system passed.

Keywords: rate limiting, tested and passed

Why Did You Choose This Technology? Questions

Q36. Why React.js for the frontend?

React lets me build the interface using small, reusable components, which makes the code easier to manage and update compared to writing repetitive plain HTML and JavaScript.

Keywords: reusable components, easier maintenance

Q37. Why Node.js and Express for the backend?

Node.js lets me use JavaScript on both the frontend and backend, which reduces context-switching and lets the whole team work with one single language across the entire project.

Keywords: single language across stack

Q38. Why MongoDB Atlas specifically, not a local MongoDB installation?

MongoDB Atlas is a managed cloud database, so I don't have to worry about server maintenance, backups, or scaling manually, and it's accessible from anywhere during development and deployment.

Keywords: managed cloud database, no manual maintenance

Q39. Why Razorpay instead of another payment gateway?

Razorpay is a widely trusted gateway with strong documentation on signature verification and good support for Indian payment methods like UPI, which matched my project's target market.

Keywords: trusted gateway, strong docs, UPI support

Q40. Why Gemini instead of ChatGPT or another AI model?

Gemini was the provider I actually integrated and tested; the way I built the prompt-handling logic is not tied to one specific AI provider, so switching providers later would not require redesigning the whole system.

Keywords: provider-agnostic design, tested with Gemini

Q41. Why the Web Speech API instead of a paid cloud speech service like Google Cloud Speech?

The Web Speech API is completely free and built into the browser itself, so I could add voice features without any additional cost or extra server infrastructure.

Keywords: free, browser-native, no extra cost

Q42. Why Socket.IO instead of plain WebSockets?

Socket.IO automatically falls back to regular HTTP requests if a WebSocket connection isn't available on a particular network, making the real-time feature more reliable across different situations.

Keywords: automatic fallback, more reliable

Q43. Why did you build two separate portals instead of one combined website?

Separating the user portal and admin portal means the simple, consumer-facing design of one doesn't get complicated by the data-heavy needs of the other, and each can be developed and secured independently.

Keywords: separation of concerns, independent security

Q44. Why Power BI instead of building your own custom charts?

Power BI is an industry-standard tool that property owners may already be familiar with, and it saves development time compared to building a custom reporting dashboard from scratch.

Keywords: industry-standard, saves development time

Q45. Why did you use JWT instead of traditional server-side sessions?

JWT is stateless, meaning the server doesn't need to store session data in memory for every logged-in user, which makes the system simpler to scale across multiple servers later.

Keywords: stateless, easier to scale

Challenges Faced Questions

Q46. What was the most difficult part of building this project?

The most challenging part was getting payment verification right, because the natural first instinct is to trust whatever the payment gateway tells you, and I had to specifically learn why that assumption is unsafe.

Keywords: payment verification, security learning curve

Q47. Did you face any challenges with the voice assistant?

Yes, ensuring the correct language was used consistently, especially when a user changed the language mid-conversation and then replayed an older message, required carefully tagging each message with the language it was actually generated in.

Keywords: language consistency, message tagging

Q48. Did you face challenges integrating Power BI?

Yes, since Power BI's scheduled refresh cannot work with a token that expires every few days like a normal user login, I had to design a separate, non-expiring API key specifically for that connection.

Keywords: token expiry mismatch, separate API key

Q49. How did you handle debugging issues during development?

I tested each module individually first, then tested the full flow end-to-end, and used structured logging to trace exactly where a request failed when something didn't behave as expected.

Keywords: modular testing, structured debugging

Q50. Did you face any performance challenges?

Yes, certain endpoints like payments and AI chat were naturally slower due to external services like Cloudinary uploads and the Gemini API round-trip, so I made sure to measure and clearly explain those delays rather than hide them.

Keywords: external service latency, transparent measurement

Q51. How did you manage security concerns during development?

I specifically designed and ran fourteen adversarial test cases targeting authentication, payments, and input handling, rather than only testing that normal, expected actions worked correctly.

Keywords: adversarial testing, not just happy-path

Q52. What was challenging about supporting three different languages?

Making sure the AI would translate conversational text but leave property names, phone numbers, and addresses untouched required carefully worded instructions to the AI model itself.

Keywords: selective translation challenge

Q53. Did you face challenges deciding what to keep in scope?

Yes, it was tempting to add many features, but I had to consciously limit scope to what could be built and tested properly, moving the rest into future scope.

Keywords: scope discipline, quality over quantity

What Improvements Can Be Made? Questions

Q54. What would you improve if you had more time?

I would upgrade the AI assistant from simple context-injection to full retrieval-augmented generation using vector search, so it stays accurate even as the property catalogue grows much larger.

Keywords: RAG upgrade, scalability of AI grounding

Q55. How would you make the admin dashboard more reliable?

I would change the current single combined database query into several independent queries, so that if one report fails, the rest of the dashboard still loads normally.

Keywords: independent queries, fault tolerance

Q56. How would you improve scalability for many more users?

I would add horizontal scaling for the server and rely on MongoDB Atlas's auto-scaling, since testing showed performance starts to degrade meaningfully beyond 150 concurrent users.

Keywords: horizontal scaling, auto-scaling database

Q57. Would you add a mobile app in the future?

Yes, a React Native mobile app is planned, and it would reuse the exact same backend APIs without needing to redesign any business logic.

Keywords: React Native, API reuse

Q58. How would you improve accessibility further?

I would extend today's voice-level multilingual support into full interface translation, including a right-to-left layout for Arabic-speaking users.

Keywords: full UI localization, RTL layout

Q59. What analytics improvements would you add?

I would add predictive analytics that forecasts rental pricing, vacancy periods, and late-payment risk using the booking and payment history the system already collects.

Keywords: predictive analytics, existing data reuse

Q60. How could the payment system be improved further?

I could add more payment gateway options beyond Razorpay, and introduce automatic refund handling for cancelled bookings.

Keywords: multiple gateways, refund automation

Q61. How would you improve testing coverage?

I would add automated regression testing that runs on every code change, rather than relying on manually scripted test scenarios executed at specific milestones.

Keywords: automated regression testing

Security Questions

Q62. What security measures does your project implement?

My project uses JWT authentication with per-request status checks, HMAC-SHA256 payment signature verification, rate limiting, input validation, and role-based access control, all tested through adversarial security cases.

Keywords: JWT, HMAC, rate limiting, RBAC

Q63. How do you prevent unauthorized access to admin features?

Every admin route checks both the user's JWT token and their role, and rejects the request entirely if the token belongs to a non-admin user or a deactivated account.

Keywords: token + role check, deactivated rejection

Q64. How do you protect against tampered payment amounts?

The cryptographic signature we recompute is based on the order and payment identifiers, not the amount field, but the amount is separately confirmed against the payment gateway's own authoritative record before being accepted.

Keywords: signature over IDs, authoritative amount check

Q65. Does your system prevent SQL injection?

Since we use MongoDB rather than SQL, traditional SQL injection does not apply, but we do specifically test and guard against similar query-injection attempts using MongoDB operators in search fields.

Keywords: NoSQL, MongoDB operator injection guarded

Q66. How do you prevent cross-site scripting, or XSS, attacks?

React automatically escapes any text rendered on the page by default, so even if a malicious script is entered into a property title, it displays as harmless plain text rather than executing.

Keywords: React auto-escaping, tested and passed

Q67. What is rate limiting and why is it important?

Rate limiting restricts how many requests a single user or IP address can make in a short time window, which protects the system from being overwhelmed by automated bots or brute-force login attempts.

Keywords: request limiting, brute-force protection

Q68. How do you validate uploaded files for security?

Uploaded files are checked for both file type and size before being accepted, and a test specifically confirmed that a disguised executable file is correctly rejected.

Keywords: file type/size validation, tested

Q69. Is user password data stored securely?

Yes, passwords are never stored in plain text; they are hashed before storage, and this was specifically verified as part of our security testing.

Keywords: password hashing, verified by test

Q70. What happens if a JWT token is stolen by an attacker?

Since every request re-checks the user's live account status, if the account is deactivated immediately after discovering the theft, the stolen token becomes useless on its very next use.

Keywords: immediate deactivation effect

Q71. Have you tested your system against real attack scenarios, or only normal use?

Yes, beyond the thirty-eight functional tests for normal use, I specifically designed and ran fourteen separate adversarial security tests simulating real attack attempts, and all fourteen were correctly blocked.

Keywords: adversarial testing, not just happy-path

Database Questions

Q72. What database did you use and why?

I used MongoDB Atlas, a cloud-hosted NoSQL database, because our property-related data has flexible, varying structures that suit a document-based model better than fixed SQL tables.

Keywords: MongoDB Atlas, NoSQL, flexible structure

Q73. What are the main collections in your database?

The main collections include Users, Properties, Bookings, Payments, Contracts, Owners, Tenants, financial transactions, service or maintenance requests, and Notifications.

Keywords: ten+ collections, one per entity

Q74. How is data linked between collections in MongoDB?

MongoDB uses references, similar to storing an ID pointing to another document, and our Mongoose layer helps join and populate this related information when needed.

Keywords: references, Mongoose population

Q75. How do you keep search fast on the Properties collection?

We created compound indexes on frequently searched fields like property type, city, and price, so the database can find matching results quickly instead of scanning every single record.

Keywords: compound indexes, faster search

Q76. How does your database support the Power BI reports?

Power BI calls dedicated aggregation endpoints where MongoDB itself performs the grouping and counting work directly, so only the final summarized numbers are sent, not raw records.

Keywords: server-side aggregation, summarized data

Q77. What would happen if your database connection fails?

Our system uses Mongoose's built-in reconnection logic with automatic retry and backoff, so it attempts to reconnect gracefully rather than crashing immediately.

Keywords: auto-reconnect, backoff logic

Q78. How do you ensure data consistency across the two portals?

Both the user portal and the admin portal talk to the exact same backend and database, so there is only one single source of truth, and no separate or duplicated copy of the data exists.

Keywords: single source of truth, shared backend

Q79. Why not use two different databases for the two portals?

Using one shared database avoids data duplication and synchronization problems, ensuring that what an admin sees always exactly matches what actually happened on the user side.

Keywords: avoids duplication, avoids sync issues

API Questions

Q80. How many main API groups does your backend expose?

The backend exposes API groups for authentication, properties, bookings, payments, purchase contracts, AI chat, admin operations, and a separate analytics API for Power BI.

Keywords: eight main API groups

Q81. What HTTP methods does your API use?

Like most REST APIs, it uses GET to retrieve data, POST to create new records like a booking or payment, PUT or PATCH to update existing records, and DELETE for removal where applicable.

Keywords: GET, POST, PUT, DELETE

Q82. How is the analytics API for Power BI different from your normal APIs?

It is protected by a completely separate, non-expiring API key rather than the normal seven-day user login token, since Power BI's scheduled refresh cannot handle a token that regularly expires.

Keywords: separate API key, non-expiring credential

Q83. How do you prevent someone from misusing the AI chat API?

The AI chat endpoint has its own dedicated rate limiter to prevent abuse, and it requires the user to be properly authenticated before accepting a request.

Keywords: rate limiter, authentication required

Q84. What does your Payments API actually do step by step?

It receives payment details from the client, but instead of trusting them directly, it independently verifies the cryptographic signature and the authoritative payment status from Razorpay before updating any record.

Keywords: receive, verify, then update

Q85. How would another developer understand how to use your APIs?

Each API endpoint follows a consistent, predictable REST naming pattern, and grouping them by feature, such as bookings or payments, makes it straightforward to understand what each one does.

Keywords: consistent naming, feature-based grouping

Q86. What happens if invalid data is sent to your API?

Input validation checks run before any data reaches the database, and invalid requests are rejected with a clear error message rather than being silently accepted or crashing the server.

Keywords: input validation, clear rejection

Q87. Can your APIs be reused for a future mobile app?

Yes, since the APIs are built as a standard REST backend independent of the web frontend, a future React Native mobile app could call the exact same APIs without any redesign.

Keywords: REST reusability, mobile-ready

Authentication Questions

Q88. How does login work in your system?

When a user logs in with correct credentials, the server creates and signs a JWT token containing the user's identity, which the browser then sends along with every future request.

Keywords: credentials checked, JWT issued

Q89. What is the difference between authentication and authorization?

Authentication means proving who you are, like logging in, while authorization means deciding what you are allowed to do once your identity is confirmed, like whether you can access admin features.

Keywords: authentication=identity, authorization=permission

Q90. How many user roles exist in your system?

There is a four-level hierarchy: Guest, Registered User, Admin, and Super Admin, where each higher level inherits the permissions of the levels below it.

Keywords: four-level role hierarchy

Q91. What happens during password reset?

The user requests a reset, receives a one-time password with an expiry time by email, and must use it within that time window before it becomes invalid.

Keywords: OTP-based reset, expiry enforced

Q92. Why do you re-check the user's status on every request instead of just trusting the login token?

Because a token stays technically valid until it expires, even if the account is blocked afterward; re-checking the live status closes this gap and enforces access changes immediately.

Keywords: closes the block-to-enforcement gap

Deployment Questions

Q93. How is your project deployed?

The backend and frontend are deployed as a cloud-hosted web application, connecting to MongoDB Atlas as the cloud database, so the whole system is accessible over the internet rather than only on a local machine.

Keywords: cloud-hosted, MongoDB Atlas

Q94. What environment variables or secrets does your project need?

Sensitive values such as the database connection string, JWT secret key, Razorpay keys, and the Gemini API key are stored as environment variables rather than being written directly into the code.

Keywords: environment variables, secrets not hardcoded

Q95. How would you scale this system for production traffic?

I would add horizontal scaling for the Node.js server behind a load balancer, combined with MongoDB Atlas's own auto-scaling, based directly on what my load testing showed was needed beyond 150 users.

Keywords: load balancer, auto-scaling

Q96. What would you monitor after deployment?

I would monitor server response times, error rates, and database performance, similar to the exact metrics I already measured during testing, such as mean latency and error percentage.

Keywords: response time, error rate monitoring

Q97. Is your system ready for production use today?

It has been functionally and security tested thoroughly and performs well at a moderate scale, though I would recommend adding automated monitoring and the fault-tolerant dashboard improvement before very large-scale production use.

Keywords: tested and capable, with noted improvements

Architecture Questions

Q98. Why did you separate the frontend into two portals instead of one?

Separating the user-facing portal from the admin portal means each can be designed for its actual audience, a simple browsing experience for users and a data-dense management experience for admins, without compromising either.

Keywords: audience-specific design

Q99. Why does neither portal talk directly to the database?

Routing every request through the backend API ensures that authentication and authorization checks always happen at one single, auditable point before any data is touched.

Keywords: single auditable gate, no direct DB access

Q100. Where does the voice assistant fit in your architecture?

It sits inside the same authenticated request flow as any other feature; voice requests still pass through the login and role check before reaching the Gemini AI service.

Keywords: same security path as other features

Q101. Why is Authentication placed before the Voice Assistant in your architecture diagram?

To show that even a conversational, voice-based request is still a protected action requiring identity verification, not a special unauthenticated shortcut.

Keywords: voice = protected, not a shortcut

Q102. How does Power BI fit into the architecture without compromising security?

It connects to a separate, dedicated analytics API secured by its own non-expiring key, completely isolated from the interactive user and admin authentication system.

Keywords: isolated analytics path

Q103. What is the biggest architectural weakness you are aware of?

The admin dashboard's reliance on one single combined query batch across eight collections, where the failure of any one query currently causes the entire dashboard request to fail.

Keywords: single batch query, admitted weakness

Q104. Could this architecture support a completely different type of business, not just property management?

Yes, the core architecture, authentication, real-time notifications, verified payments, and AI assistance, is fairly generic and could be adapted to other transaction-heavy service businesses with moderate changes.

Keywords: generic, adaptable architecture

Q105. What made you choose a three-tier architecture instead of microservices?

At this project's scale, a three-tier monolith is simpler to build, test, and deploy; microservices would add operational complexity without a corresponding benefit at this size.

Keywords: simplicity over premature complexity

Part D — Final Revision Sheet

One-Page Quick Revision

PropManage is a MERN stack web application (MongoDB, Express, React, Node) that unifies property listing, booking, payments, contracts, and maintenance for small and mid-size property businesses. It has a dual-portal design: a client-user portal and a client-admin portal, both talking to one shared Node.js/Express backend and one MongoDB Atlas database. Real-time admin alerts use Socket.IO. Payments are verified using an HMAC-SHA256 signature recomputed on the server, never trusted from the client. The AI assistant uses Google's Gemini API combined with the browser's free Web Speech API to support voice input and output in English, Hindi, and Telugu, grounded in live property data. A separate, non-expiring API key connects the system to Power BI for live business dashboards. Testing showed 38 out of 38 functional tests passed, 14 out of 14 security tests passed, and the system sustained 100 concurrent users at under 500 milliseconds mean latency with under 0.2 percent errors. Known limitations include the admin dashboard's single-batch query design and the AI's simpler context-assembly grounding instead of full retrieval-augmented generation. Future work includes a mobile app, full UI localisation, predictive analytics, and IoT integration.

Important Keywords to Memorize

MERN stack, dual-portal architecture, three-tier architecture, JWT authentication, role-based access control (RBAC), HMAC-SHA256 payment verification, Socket.IO real-time notification, Gemini API, Web Speech API, multilingual voice assistant (English/Hindi/Telugu), context-grounded AI, Power BI analytics API, MongoDB Atlas, Mongoose, compound indexing, Razorpay, functional testing (38/38), adversarial security testing (14/14), concurrent load testing (100 users, sub-500ms), retrieval-augmented generation (RAG) as future work, React Native as future work.

Common Mistakes to Avoid During Viva

Do not claim your project has zero limitations — examiners trust students more when they honestly acknowledge a limitation, like the dashboard's single-batch query issue, and explain how they would fix it. Do not confuse authentication with authorization; they are different concepts. Do not say MongoDB has “tables” — it has collections and documents. Do not claim your voice assistant uses full RAG, or retrieval-augmented generation — be clear that today it uses a simpler context-assembly method, and RAG is future work. Do not say your system is completely unhackable — say it passed fourteen specific adversarial tests, which is a precise, defensible claim rather than an absolute one. Finally, do not read answers word for word from memory in a robotic tone; speak naturally, as if explaining to a curious friend.

Tips to Answer Confidently If You Forget an Answer

If you blank out on a question, stay calm and start by restating the question in your own words, which buys you a few seconds to think. If you genuinely don't remember an exact number, it is perfectly acceptable to say, “I don't remember the exact figure right now, but the trend showed that performance was strong up to a certain point and required scaling beyond it,” which shows understanding even without a precise number. You can also redirect confidently to a slide or diagram you do remember well, such as the System Architecture diagram, and use it to reconstruct your answer step by step. Never guess a fake number or invent something you are unsure of — examiners respect “I'm not fully certain, but here is my reasoning” far more than a confidently wrong answer.

Above all, remember that you built and tested this project yourself, so the knowledge is already inside you; your job in the viva is simply to speak it out loud, calmly and clearly.